17. Implement Sharding and Partitioning

Key Terms:

Term	Full Name	What it does
Shard	Database partition	Horizontal slice of data on separate server
Shard Key	Partition key	Field used to determine which shard holds data
Consistent Hashing	Distribution algorithm	Minimizes data movement when adding/removing nodes
Hot Spot	Uneven load	One shard receiving disproportionate traffic
Scatter-Gather	Query pattern	Query all shards, merge results
Resharding	Rebalancing	Moving data when adding/removing shards

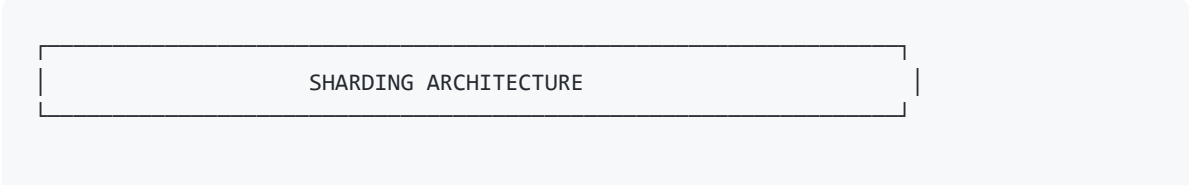
Problem Statement

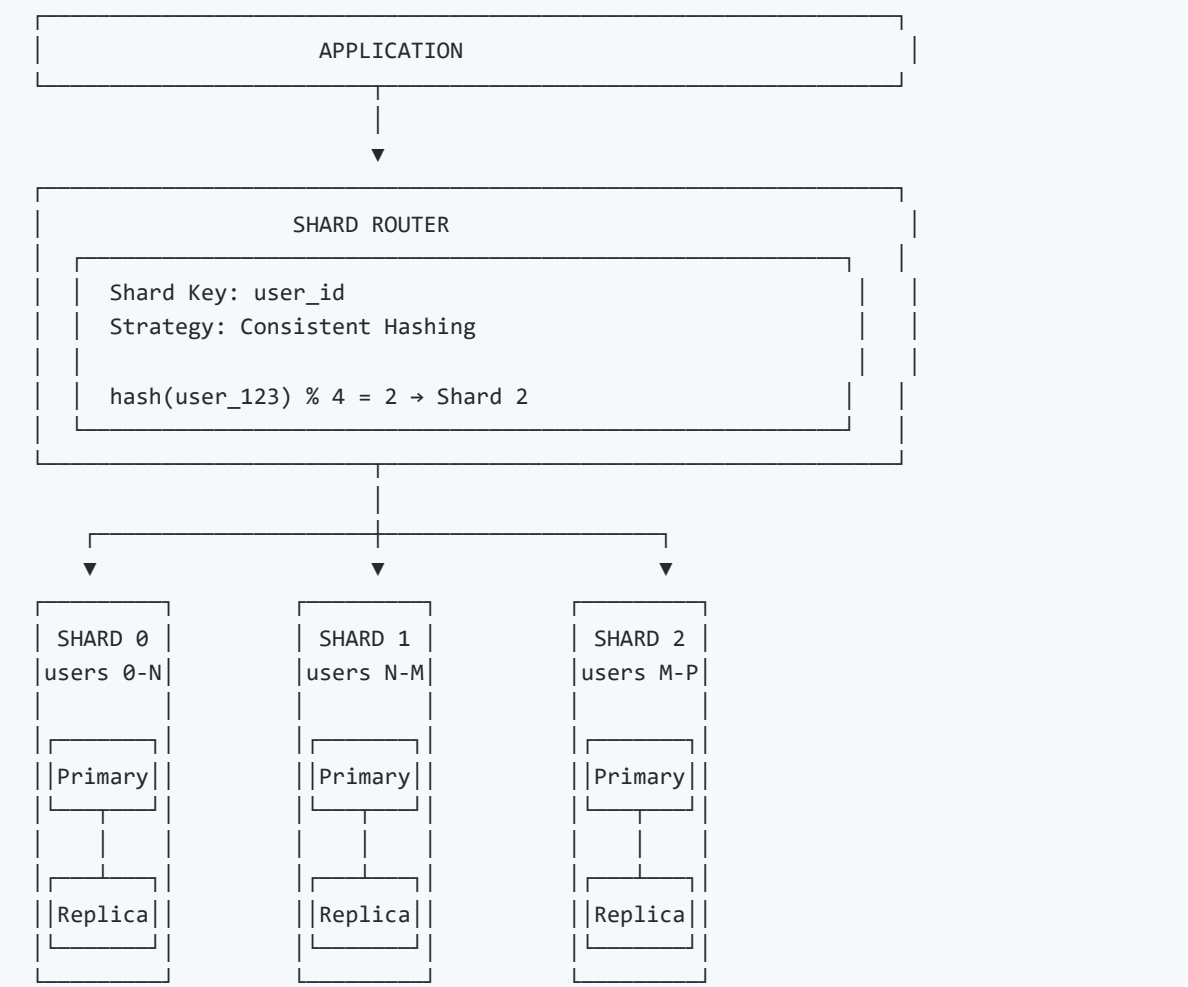
Design a data sharding strategy for a large-scale database that distributes data across multiple servers while maintaining query efficiency.

Clarifying Questions to Ask

1. **Data size**: How much data? Growth rate?
2. **Query patterns**: Point lookups, range queries, aggregations?
3. **Access patterns**: Even distribution or hot spots?
4. **Consistency**: Strong or eventual consistency?
5. **Resharding**: How to add/remove nodes?
6. **Cross-shard**: Queries spanning multiple shards needed?
7. **Technology**: SQL or NoSQL database?

High-Level Design





API Design

Shard Management APIs

Method	Endpoint	Description
`GET`	`/api/v1/shards`	List all shards and their status
`POST`	`/api/v1/shards/split`	Split a shard
`POST`	`/api/v1/shards/merge`	Merge two shards
`GET`	`/api/v1/shards/{shardId}/stats`	Get shard statistics (size, QPS)

Routing API (Internal)

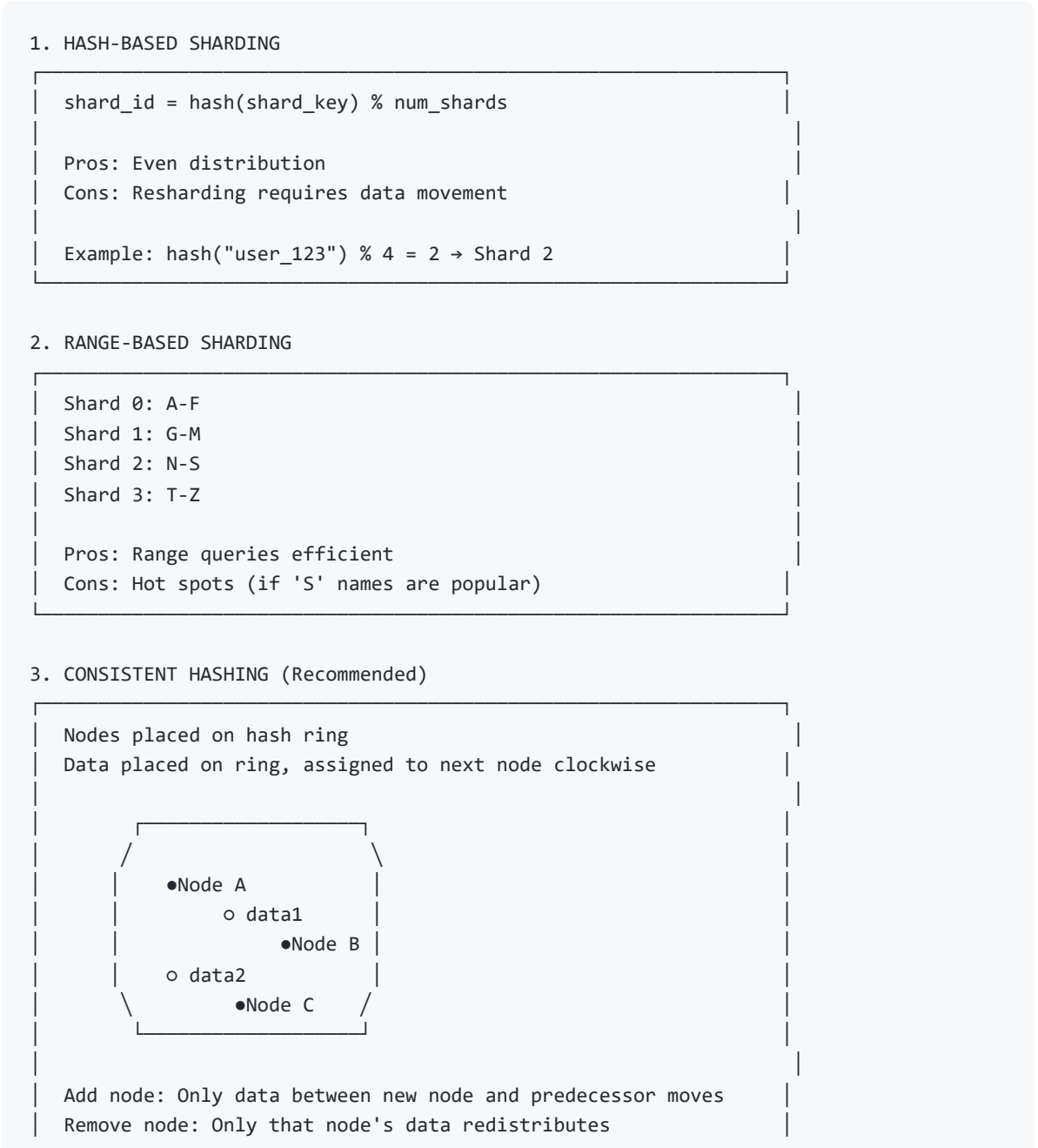
Method	Endpoint	Description
`GET`	`/api/v1/route?key={shardKey}`	Resolve shard for a given key

`GET`	`/api/v1/mapping`	Get full shard mapping table

Resharding APIs

Method	Endpoint	Description
`POST`	`/api/v1/reshard`	Initiate resharding operation
`GET`	`/api/v1/reshard/{jobId}/status`	Get resharding progress
`POST`	`/api/v1/reshard/{jobId}/cancel`	Cancel resharding

Sharding Strategies



```
└──────────────────────────────────────────────────────────────┘

4. DIRECTORY-BASED SHARDING
┌──────────────────────────────────────────────────────────────┐
│  Lookup table maps keys to shards                            │
│                                                              │
│  user_123 → Shard 2                                          │
│  user_456 → Shard 0                                          │
│  user_789 → Shard 1                                          │
│                                                              │
│  Pros: Flexible, can handle hot spots                        │
│  Cons: Directory is single point of failure                  │
└──────────────────────────────────────────────────────────────┘
```

Choosing a Shard Key

```
GOOD SHARD KEYS:
┌──────────────────────────────────────────────────────────────┐
│  ✓ user_id - Even distribution, most queries are per-user    │
│  ✓ tenant_id - Multi-tenant SaaS, queries within tenant      │
│  ✓ order_id - E-commerce, orders are independent             │
│  ✓ Compound: (tenant_id, user_id) - Keeps tenant data together │
└──────────────────────────────────────────────────────────────┘

BAD SHARD KEYS:
┌──────────────────────────────────────────────────────────────┐
│  ✗ timestamp - All recent data on one shard (hot spot)       │
│  ✗ country - Uneven (US shard overloaded)                    │
│  ✗ status - Few values, poor distribution                    │
│  ✗ auto-increment ID - Sequential writes to one shard        │
└──────────────────────────────────────────────────────────────┘
```

Cross-Shard Operations

```
SCATTER-GATHER PATTERN:
┌──────────────────────────────────────────────────────────────┐
│  Query: SELECT * FROM orders WHERE total > 1000              │
│  (No shard key in WHERE clause)                              │
│                                                              │
│  1. Router sends query to ALL shards (scatter)               │
│  2. Each shard executes query locally                        │
│  3. Router merges results (gather)                           │
│                                                              │
│  Performance: O(num_shards) - avoid if possible              │
└──────────────────────────────────────────────────────────────┘

GLOBAL TABLES:
┌──────────────────────────────────────────────────────────────┐
│  Small, rarely-changing tables replicated to all shards      │
│  Example: countries, currencies, config                      │
```

```
│                                                                 │
│  Allows local joins without cross-shard queries                 │
└─────────────────────────────────────────────────────────────────┘
```

Resharding Strategy

```
ONLINE RESHARDING (Zero-downtime):

┌─────────────────────────────────────────────────────────────────┐
│  1. Add new shards to cluster                                   │
│  2. Start dual-writes to old and new locations                  │
│  3. Background migration of existing data                       │
│  4. Verify data consistency                                     │
│  5. Switch reads to new shard mapping                           │
│  6. Stop writes to old locations                                │
│  7. Remove old shard assignments                                │
│                                                                 │
│  Tools: Vitess (MySQL), Citus (PostgreSQL)                      │
└─────────────────────────────────────────────────────────────────┘
```

Trade-offs

Decision	Option A	Option B	Recommendation
Strategy	Hash	Range	**Hash** - better distribution
Shard key	Single	Compound	**Compound** - more flexibility
Cross-shard	Allow	Avoid	**Design to avoid** - better perf
Resharding	Offline	Online	**Online** - zero downtime

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Consistent Hashing	Minimize data movement when adding/removing shards	Tech Guide

How to Explain It Well

> **Opening**: "Sharding distributes data across servers for scale. I'll use consistent hashing for even distribution and minimal resharding impact."

> **Key talking points**:
>
> - "Consistent hashing: adding a node moves only 1/N of data"
> - "Choose shard key based on query patterns - usually user_id"
> - "Avoid cross-shard queries - design schema to keep related data together"
> - "Global tables for small reference data to enable local joins"
> - "Online resharding with dual-writes for zero downtime"
